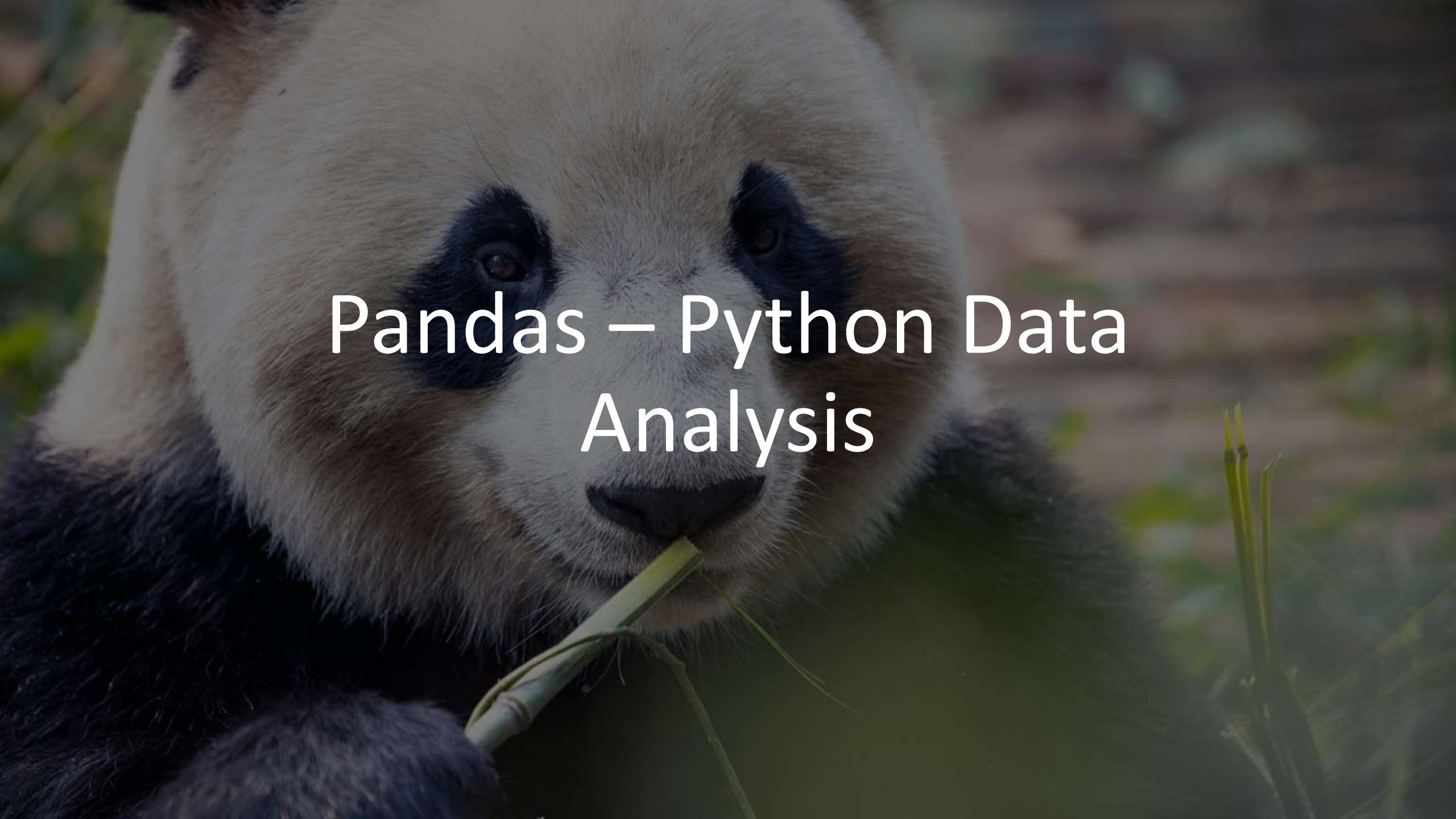


Pandas – Python Data Analysis

What is Pandas?

Pandas is a Python library used for working with data sets.
It has functions for analyzing, cleaning, exploring, and manipulating data.
The name "Pandas" has a reference to both "Panel Data", and "Python Data Analysis" and was created by Wes McKinney in 2008.

Why Use Pandas?

Pandas allows us to analyze big data and make conclusions based on statistical theories.
Pandas can clean messy data sets, and make them readable and relevant.
Relevant data is very important in data science.
:}

Data Science: is a branch of computer science where we study how to store, use and analyze data for deriving information from it.

What Can Pandas Do?

Pandas gives you answers about the data. Like:
Is there a correlation between two or more columns?
What is average value?
Max value?
Min value?
Pandas are also able to delete rows that are not relevant, or contains wrong values, like empty or NULL values. This is called *cleaning* the data.

Where is the Pandas Codebase?

The source code for Pandas is located at this github repository https://github.com/pandas-dev/pndas

Why Pandas for Tabular Data?

- Tables contain rows and columns.
- Pandas provides DataFrame to represent tables.
- It supports different data types in different columns.
- Data can be selected, filtered, sorted and modified easily.
- Pandas can read data from CSV, Excel and other sources.

What is Pandas?

- Pandas is a Python library for data manipulation and analysis.
- Used to work with structured and tabular data.
- Commonly used in Data Science, Machine Learning and Business Analytics.
- Import using: import pandas as pd



Why Use Pandas?

- Easy handling of tabular data
- Read and write CSV and Excel files
- Filtering, sorting and grouping data
- Handle missing values
- Perform statistical analysis
- Works well with NumPy and Matplotlib

Main Data Structures

SERIES – ONE-DIMENSIONAL LABELED DATA. SERIES CAN BE CONSIDERED AS A SINGLE COLUMN OF DATA.

DATAFRAME – TWO-DIMENSIONAL TABULAR DATA. DATAFRAME IS SIMILAR TO A SPREADSHEET OR DATABASE TABLE.DATAFRAME CAN CONTAIN MULTIPLE SERIES ASCOLUMNS.



Pandas Series

- A Series is a one-dimensional labeled data structure.
- Example:

marks = pd.Series([85, 90, 78, 92])

Default index: 0, 1, 2, 3



Pandas Series

A Pandas Series is like a column in a table.

It is a one-dimensional array holding data of any type.

Example

Create a simple Pandas Series from a list:

```
import pandas as pd
a = [1, 7, 2]
myvar = pd.Series(a)
print(myvar)
```

Output:

```
0    1
1    7
2    2
dtype: int64
```

Series with Custom Index

marks = pd.Series([85, 90, 78], index=["Anu", "Binu", "Cathy"])

marks["Anu"] → 85

loc accesses by label; iloc accesses by integer position

```
import pandas as pd
marks = pd.Series( [85, 90, 78],
 index=["Anu", "Binu", "Cathy"] ) print(marks)
```

```
Anu 85
Binu 90
Cathy 78
dtype: int64
```

```
print(marks["Anu"])
```

Output:

85

Data Frame Structure

- Rows represent individual records.
- Columns represent attributes or variables.
- Each row has an index.

A Data Frame consists of:

- **Index** – identifies each row.
- **Columns** – represent variables or fields.
- **Data** – values stored in rows and columns.
- **Data Types** – each column can have a different data type.

Example: Name, Age and Mark are columns.

Creating a DataFrame

```
data = {"Name":["Anu","Binu","Cathy"],
 "Age":[20,21,19],
"Mark":[85,90,78]}
df = pd.DataFrame(data)
```

A DataFrame contains rows and columns.

```python
import pandas as pd

data = {
    "Name": ["Arun", "Ravi", "Anu"],
    "Age": [20, 21, 20],
    "Mark": [85, 90, 88]
}

df = pd.DataFrame(data)

print(df)
```

Output:

```
   Name  Age  Mark
0  Arun   20    85
1  Ravi   21    90
2   Anu   20    88
```

Property	Purpose
`df.shape`	Returns number of rows and columns
`df.columns`	Displays column names
`df.index`	Displays row index
`df.dtypes`	Displays data types
`df.size`	Returns total number of elements
`df.ndim`	Returns number of dimensions

Viewing Data

- df.head() – first five rows
- df.head(2) – first two rows
- df.tail() – last five rows
- Useful for quickly inspecting a dataset.

Function	Purpose
`df.head()`	View first 5 rows
`df.tail()`	View last 5 rows
`df.columns`	View column names
`df["Name"]`	View one column
`df[["Name","Mark"]]`	View multiple columns
`df.iloc[]`	Access by position
`df.loc[]`	Access by index label
`df.info()`	View DataFrame information
`df.describe()`	View statistical summary

Accessing Columns and Rows

In **Pandas**, we can access specific **columns** and **rows** of a Data Frame using column names, .loc[], and .iloc[].

1. Accessing a Column

Suppose we have:

```python
import pandas as pd

data = {
    "Name": ["Anu", "Binu", "Cathy"],
    "Age": [20, 21, 19],
    "Marks": [85, 90, 78]
}

df = pd.DataFrame(data)
print(df)
```

Output:

	Name	Age	Marks
0	Anu	20	85
1	Binu	21	90
2	Cathy	19	78

Access one column

Python

```python
print(df["Name"])
```

Output:

```
0      Anu
1     Binu
2    Cathy
Name: Name, dtype: object
```

You can also use:

Python

```python
print(df.Age)
```

Note: df["Age"] is generally preferred because it also works when column names contain spaces or conflict with Data Frame methods.

Accessing Multiple Columns

Use a list of column names:

```
print(df[["Name", "Marks"]])
```

Output:

	Name	Marks
0	Anu	85
1	Binu	90
2	Cathy	78

Accessing Rows Using .loc[]

.loc[] accesses rows using **labels/index values**.

Access one row

```
print(df.loc[0])
```

Output:

```
Name     Anu
Age       20
Marks     85
```

Access multiple rows

```
print(df.loc[0:1])
```

Output:

	Name	Age
0	Anu	20
1	Binu	21

Accessing Rows Using .iloc[]

.iloc[] accesses rows using **integer positions**.

```
print(df.iloc[0])
```

This accesses the **first row**.

Python

```
print(df.iloc[1])
```

This accesses the **second row**.

Multiple rows

Python

```
print(df.iloc[0:2])
```

This accesses the first two rows.

Important difference:

- `.loc[]` → uses **index labels**
- `.iloc[]` → uses **integer positions**

Accessing a Specific Cell Using .loc[]

Using `.loc[]`

Python

```
print(df.loc[1, "Marks"])
```

Output:

```
90
```

Using `.iloc[]`

Python

```
print(df.iloc[1, 2])
```

Output:

```
90
```

Here, `1` represents the second row and `2` represents the third column.

Adding and Removing Columns

Note: axis=1 refers to columns; axis=0 refers to rows

Operation	Code
Add column	`df["Grade"] = ["A", "A+", "B"]`
Add calculated column	`df["Bonus"] = df["Mark"] + 5`
Remove one column	`df.drop("Grade", axis=1)`
Remove multiple columns	`df.drop(["Grade", "Bonus"], axis=1)`
Remove using `del`	`del df["Grade"]`
Remove using `pop()`	`df.pop("Grade")`

Filtering Data

Filtering data means selecting only the rows that satisfy a particular **condition**.

It is useful when we want to find specific records from a dataset.

	Name	Age	Marks
0	Anu	20	85
1	Binu	21	90
2	Cathy	19	78
3	David	22	65

Example Dataset

```
import pandas as pd
 data = { "Name": ["Anu", "Binu", "Cathy", "David"], "Age": [20,
21, 19, 22], "Marks": [85, 90, 78, 65] }
df = pd.DataFrame(data)
print(df)Fil
```

Filtering Using a Condition

- To find students who scored more than 80:

```
print(df[df["Marks"] > 80])
```

Output:

	Name	Age	Marks
0	Anu	20	85
1	Binu	21	90

Filters students whose marks are greater than 80.

Conditions can be combined for more complex filtering.

2. Filtering Equal Values

Find students whose age is 20:

```python
print(df[df["Age"] == 20])
```

---

3. Filtering Less Than a Value

Find students who scored less than 80:

```python
print(df[df["Marks"] < 80])
```

---

4. Filtering Using Multiple Conditions

AND condition

Find students who are older than 19 **and** scored more than 80:

```python
print(df[(df["Age"] > 19) & (df["Marks"] > 80)])
```

Use `&` for **AND**.

Sorting Data

Sorting is the process of arranging data in a particular order. In Pandas, we can sort data in ascending or descending order using the `sort_values()` and `sort_index()` methods.

df.sort_values("Mark") – ascending order

df.sort_values("Mark", ascending=False) – descending ordeR

Operation	Pandas syntax
Ascending sort	`df.sort_values(by="Marks")`
Descending sort	`df.sort_values(by="Marks", ascending=False)`
Sort by multiple columns	`df.sort_values(by=["Age", "Marks"])`
Sort text alphabetically	`df.sort_values(by="Name")`
Sort index	`df.sort_index()`
Sort original DataFrame	`df.sort_values(by="Marks", inplace=True)`

```python
import pandas as pd

data = {
    "Name": ["Anu", "Binu", "Cathy", "David"],
    "Age": [20, 21, 19, 22],
    "Marks": [85, 90, 78, 65]
}

df = pd.DataFrame(data)
print(df)
```

Sorting in Ascending Order

Ascending order arranges values from smallest to largest, or alphabetically from A to Z.

Example: Sort students by marks in ascending order.

```
import pandas as pd

data = {
    "Name": ["Anu", "Binu", "Cathy", "David"],
    "Age": [20, 21, 19, 22],
    "Marks": [85, 90, 78, 65]
}

df = pd.DataFrame(data)
print(df)
```

```
print(df.sort_values(by="Marks"))
```

Sorting in Descending Order

Descending order arranges values from largest to smallest, or alphabetically from Z to A.

Example: Sort students by marks in descending order.

```
print(df.sort_values(by="Marks", ascending=False))
```

Sorting by Multiple Columns

We can sort a DataFrame using more than one column.

Example: Sort by age first, then by marks.

```
print(df.sort_values(
    by=["Age", "Marks"],
    ascending=[True, False]
))
```

Basic Statistical Functions

- mean() – average
- max() – maximum value
- min() – minimum value
- sum() – total
- count() – number of values
- std() – standard deviation

describe()

df.describe() provides a statistical summary.

Includes count, mean, standard deviation, minimum and maximum.

Also provides quartile information for numerical columns

```
import pandas as pd

data = {
    'Age': [20, 21, 22, 23, 24],
    'Marks': [70, 80, 75, 90, 85]
}

df = pd.DataFrame(data)

print(df.describe())
```

Output

```
             Age       Marks
count    5.000000    5.000000
mean    22.000000   80.000000
std      1.581139    7.905694
min     20.000000   70.000000
25%     21.000000   75.000000
50%     22.000000   80.000000
75%     23.000000   85.000000
max     24.000000   90.000000
```

Handling Missing Values

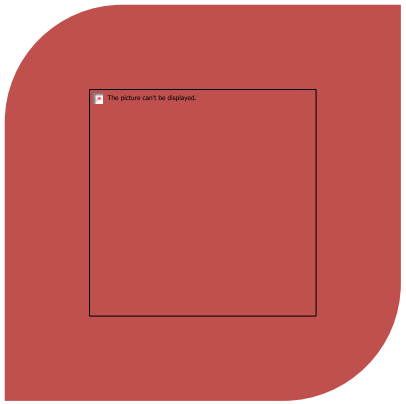
isnull() – identifies missing values

isnull().sum() – counts missing values

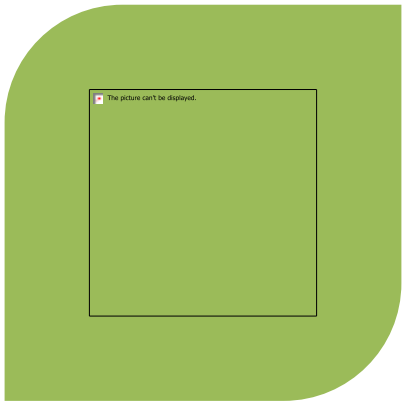
dropna() – removes rows containing missing values

fillna(value) – replaces missing values

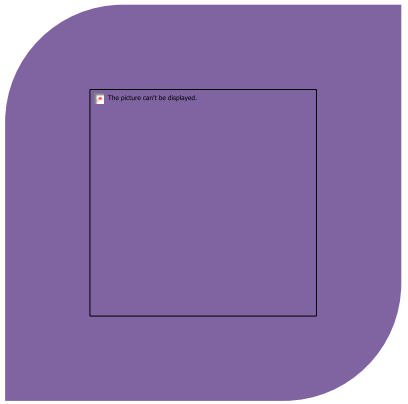
Reading CSV Files



DF = PD.READ_CSV("STUDENTS.CSV")



CSV FILES ARE COMMONLY USED FOR STORING TABULAR DATA.



DF.HEAD() CAN BE USED TO INSPECT THE IMPORTED DATA.

Writing CSV Files

- df.to_csv("students_output.csv", index=False)
- index=False prevents the DataFrame index from being saved as a separate column.

Grouping Data

- df.groupby("Department")["Mark"].mean()
- groupby() divides data into groups.
- Useful for calculating group-wise totals, averages and counts.

Pandas and NumPy

- NumPy – numerical computing and arrays
- Pandas – data manipulation and analysis
- Pandas uses NumPy internally and works closely with it.

Important Pandas Methods

- Series(), DataFrame(), head(), tail()
- loc[], iloc[], drop(), sort_values()
- isnull(), dropna(), fillna(), groupby()
- read_csv(), to_csv(), read_excel(), to_excel()

Simple Example

- import pandas as pd
- data = {"Name":["Anu","Binu","Cathy"], "Mark":[85,92,78]}
- df = pd.DataFrame(data)
- print(df["Mark"].mean())
- print(df[df["Mark"] > 80])

Advantages of Pandas

- Easy to learn and use
- Efficient tabular data handling
- Powerful filtering and sorting
- Missing-data support
- CSV and Excel support
- Useful for Data Science and Machine Learning